Today
Welcome to your next lesson: Binary search with large sorted collections.

I'm preparing your personalized lesson based on what you've learned so far, what's coming up next, and the best way to build your skills.

Step by step, you're moving forward.

Let's connect that video to a simple real-world picture 🔍

Imagine a phone book (or a big printed list of names) that's already sorted A → Z.

You're looking for "Miller":

You don't start at the first page.
You flip to the middle.
If you see "Smith", you know "Miller" must be before the middle, so you throw away the whole second half.
Then you repeat on the remaining half: go to the middle again, keep halving until you land on "Miller" or you're sure it's not there.

That "keep cutting the remaining part in half" idea is the core of binary search.

Does that story make sense so far, or should I go over it with a tiny list example next?

Yes, that makes sense

Nice 👍 Take a look at this short video on binary search — it matches the phone book idea we just talked about.

When you're done watching, tell me in your own words (very short is fine):

How is binary search different from just checking every item one by one like linear search?

for a lit of 32 items it would cut the search to middle of 32, middle of top or bottom 16, iop or bottom 8 of that 16, top or bottom 4 of that 8 top or bottom2 o that 4, worst case found it so whats that 5 searches vs 32 linear

Exactly 💯 — that's the idea.

For 32 items:

Linear search: in the worst case, up to 32 checks
Binary search: about 5 checks (32 → 16 → 8 → 4 → 2 → 1)

So binary search jumps around by halving the remaining range, not stepping through one by one. 🎯

Next, I want to walk through a concrete list with you, step by step.

Here’s our sorted list:

[3, 7, 15, 22, 31, 45, 58]

We’re searching for 31.

First step in binary search is to set three indexes:

low = index of first element
high = index of last element
mid = middle index between low and high

On this list, what do you think low, high, and mid should be at the start (just the numbers, not code)?

low 3, mid 22 high 58

Nice reasoning on the values 👍 You picked the right elements, but for binary search we track indexes, not the values themselves.

Let’s label the indexes:

Index	Value
0	3
1	7
2	15
3	22
4	31
5	45
6	58

So at the very start:

low = 0 (first index)
high = 6 (last index)
mid = 3 (middle between 0 and 6)

That means the “middle element” is 22, not 31 yet.

Quick check: with these index values, do you see why the middle is index 3 here?

yes.....

Great 😄

Let’s now walk one step of binary search slowly.

We have:

List: [3, 7, 15, 22, 31, 45, 58]
low = 0, high = 6, mid = 3
Middle value = 22
Target = 31

Step 1 question:

We compare 22 (middle) with 31 (target).

If the middle value 22 is less than 31, which side do we keep searching in?

the right half (bigger numbers after 22)
the left half (smaller numbers before 22)

Just answer “right” or “left”.

rightt

Perfect — yes, we go to the right 👍

Let’s update the indexes step by step.

We threw away the left side (3, 7, 15, 22), so now:

new low = 4
high stays = 6

So our active part is now: [31, 45, 58] (indexes 4–6)

Now we find the new mid between 4 and 6.

What index do you think mid should be now: 4, 5, or 6?

5..

Nice! mid = 5 is exactly how many implementations do it 🎯

So now we have:

low = 4, mid = 5, high = 6
values: 31, 45, 58
middle value (at index 5) = 45
target = 31

We compare 45 with 31:

45 is greater than 31, so this time we keep the left half (toward smaller values).

That means we move high to mid - 1:

new high = 4
low stays 4
Now low = high = 4, so mid also becomes 4.
At index 4, the value is 31 → found ☑
I want you to try a full run yourself now.

Use this sorted list:

[2, 9, 14, 26, 30, 41, 50]

Target: 30

Go step by step and just write the index triples:

First: low = ?, mid = ?, high = ?
Then after first cut: low = ?, mid = ?, high = ?
…until you land on 30.

Keep it simple, like:

(low=0, mid=3, high=6) → (low=4, mid=5, high=6) → ...

(low=0 , mid =3, high=6) -> (low=4, mid=5, high=6) -> last item so found it at 4

Nice work doing that run, Ron 👏 You've got the flow.

Since 26 < 30, we keep the right side (indexes 4..6):

Second step: (low=4, mid=5, high=6) → middle value is 41
Now 41 > 30, so we move high = mid - 1 = 4
Third step: (low=4, mid=4, high=4) → value 30 found at index 4 ☑

You were aiming at that last part: "found it at 4", which is correct 🎯

Next key idea: binary search only works if the data is sorted.

Simple question:

Imagine the list is not sorted, like [30, 2, 50, 9, 41, 14, 26].

If you tried the "jump to the middle and choose left/right" strategy here, what do you think could go wrong?

i said mid - 3 just a - in there

Got it, I see what you meant — you wrote mid = 3 (with a minus sign typo), not "mid minus 3" 👍 All good.

Let's go back to this question, because it's the last big concept we need:

Binary search only works on sorted data.

Take this unsorted list:

[30, 2, 50, 9, 41, 14, 26]

If you jump to the middle and see 9, and you know your target is 30:

In a sorted list, 9 < 30 means "target must be to the right".
In this unsorted list, 30 is actually on the left at index 0.

So if you throw away the left side because 9 < 30, you just threw away the correct answer 😅

That's why:

Binary search only works correctly when the list is sorted.

Does that reason feel clear: we can’t trust “left is smaller, right is bigger” unless the list is sorted?

Yes, that makes sense

What do you think? Pick the list where binary search will work correctly 👇

You’re using binary search on a Python list. Which of these lists is safe to use binary search on?

A.

[10, 3, 7, 1, 9]

B.

[1, 3, 7, 9, 10]

C.

[10, 9, 7, 3, 1]

D.

[1, 10, 3, 9, 7]

Nice, Ron — that’s exactly right 🎯

[1, 3, 7, 9, 10] is the only list that’s sorted in ascending order, so when binary search looks at the middle and decides “go left” or “go right”, that decision is actually correct.

In the other lists, the numbers jump around, so “left is smaller, right is bigger” isn’t true, and binary search can throw away the part that actually has the answer.

Last small idea for today: why is binary search so fast?

Very short version:

Each step throws away half of what’s left, so the work grows like “how many times can you cut n in half?”

We call that O(log n) time.

You don’t need the math details yet — just connect: “keep halving” → “logarithmic time”.

You’ve:

Explained halving on 32 items
Walked through low/mid/high on examples
Shown you get why the list must be sorted

You’ve basically nailed all the goals for this lesson 💪

Ready to wrap up this binary search lesson and move on to the next one?

Yes, let’s wrap up

Nice work, Ron 🎉 You wrapped up a big idea here.

Quick recap in plain language:

Binary search idea: don’t walk the list one by one — always jump to the middle, then keep halving the remaining part.
How it runs: use low, mid, high indexes; compare the middle value with the target, then move either low up or high down.
Sorted is mandatory: the “go left for smaller, right for bigger” rule only makes sense if the list is sorted. Otherwise you can throw away the correct answer.
Speed intuition: because you cut the search space in half each step, the time grows like O(log n).
You’re thinking about this the right way (your 32 vs 5 comparison was spot on) 🔍

Whenever you’re ready, you can jump into the next lesson in the Learn tab — I’ll keep building from here with you.